

AmigaED 4.0 Quick Manual

Kurzanleitung & Schnellreferenz



AmigaED ist eine plattformübergreifende C/C++- und m68k-Assembler-IDE für die klassische Amiga-Entwicklung.

Inhaltsverzeichnis

3	Einführung	3
4	Zwei Wege, deinen Code zu bauen	4
5	Modus 1: Ad-hoc-Einzeldatei-Kompilierung	5
6	Modus 2: Projektgesteuerte Kompilierung	6
7	Der Einstellungs-Editor (Prefs)	7
8	Das Kontextmenü des Editors	8
9	Empfohlene Compiler- & Linker-Schalter	9
10	Empfehlungen für Amiga-C-Programmierer	10



AmigaED

Zwei Wege zum Compilieren

Eine kurze, bebilderte Anleitung zu Ad-hoc-Einzeldatei-Kompilierung,
projektgesteuerter Kompilierung und dem Einstellungs-Editor

AmigaED ist eine plattformübergreifende C/C++- und m68k-Assembler-IDE für die klassische
Amiga-Entwicklung.

Hinweis: Die Bildschirm-Beschriftungen in dieser Anleitung sind auf Englisch (AmigaEDs Standardsprache) abgebildet.
Die Anwendung selbst besitzt aber auch eine deutsche Oberfläche (View → GUI Language).

Zwei Wege, deinen Code zu bauen

AmigaED unterstützt zwei unabhängige Wege, um aus deinem Quellcode ein lauffähiges AmigaOS-Programm zu machen. Beide stehen immer gleichzeitig zur Verfügung – nutze einfach den, der gerade zu deiner Aufgabe passt.

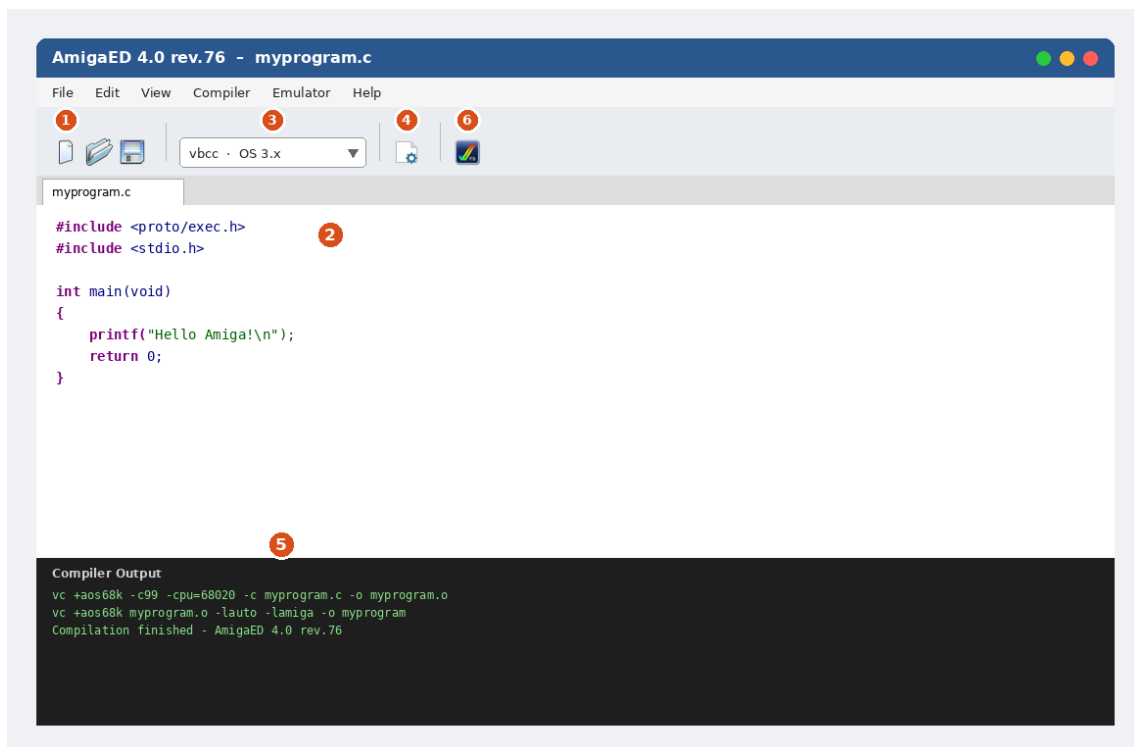
1. Ad-hoc-Einzeldatei-Kompilierung – eine einzelne Quelldatei öffnen und direkt compilieren, ganz ohne Projekt-Einrichtung. Ideal für einen schnellen Test, ein Einzelwerkzeug oder zum Ausprobieren einer Idee.

2. Projektgesteuerte Kompilierung – mehrere Dateien (Quellcode, Header, Assembler, Icons, ...) in einem Projekt zusammenfassen. AmigaED verfolgt sie, erzeugt automatisch Makefiles und baut alles mit einem Klick. Ideal für alles mit mehr als einer Datei, eine GUI (ReAction/MUI), oder Programme, an denen du später weiterarbeiten willst.

	Ad-hoc	Projekt
Geeignet für	Schnelle Einzeldatei-Tests	Mehrdatei-/GUI-Programme
Einrichtung nötig	Keine – Datei einfach öffnen	Projekt anlegen oder importieren
Mehrere Dateien verknüpft	Nein	Ja
Makefiles	Werden nicht erzeugt	Automatisch (gcc, vbcc, SAS/C)
Bleibt zwischen Sitzungen erhalten	Nein	Ja (.aep-Projektdatei)

Modus 1: Ad-hoc-Einzeldatei-Kompilierung

Nutze diesen Modus, wenn du nur eine einzelne Quelldatei schreiben und testen willst – ohne Projekt, ohne mehrere verknüpfte Dateien, ohne vorherige Einrichtung.



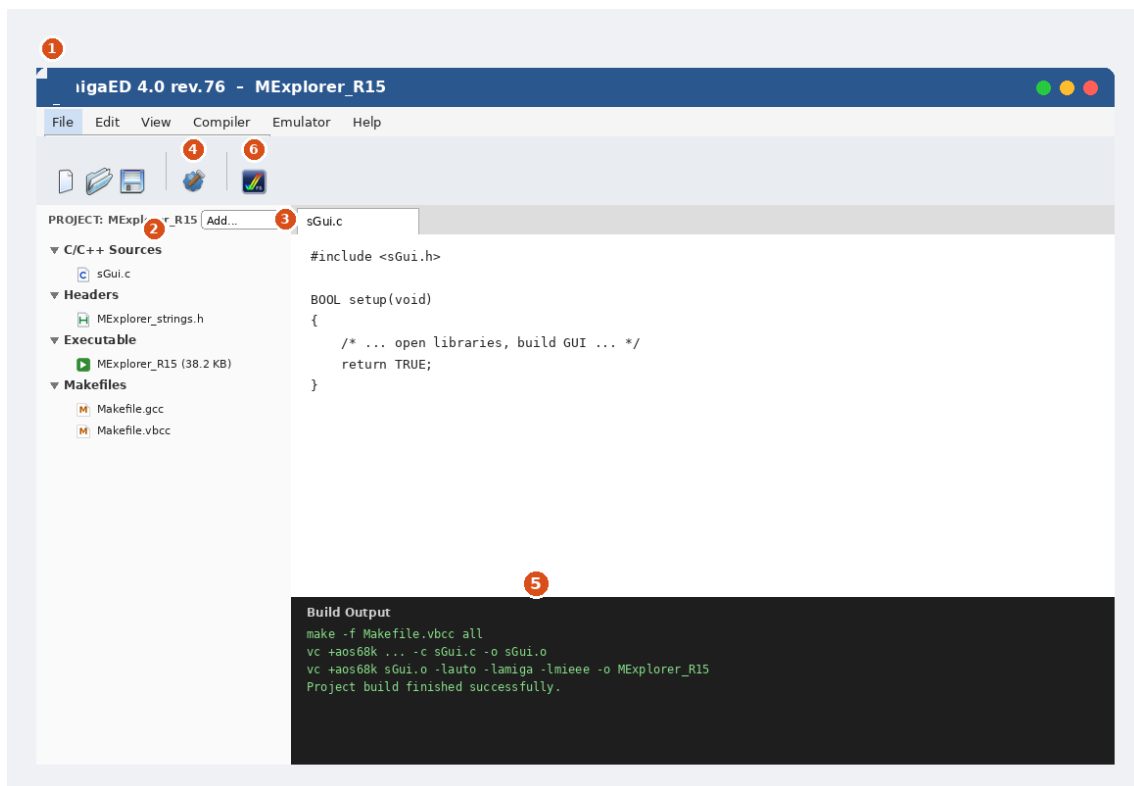
Der Editor im Ad-hoc-Modus: Die Nummern entsprechen den Schritten unten.

- 1 File** → **New** (oder **Open...**), um eine einzelne .c-/.cpp-Datei zu erstellen oder zu öffnen. Es ist kein Projekt beteiligt.
- 2** Code im Editor-Tab schreiben oder bearbeiten – mit vollständigem Syntax-Highlighting.
- 3** In der Toolbar über das Compiler-Dropdown die gewünschte **Toolchain und Ziel-OS** auswählen (z. B. vbcc / OS 3.x, oder m68k-amigaos-gcc / OS 1.3).
- 4** Auf das **Compile**-Symbol in der Toolbar klicken (oder das Compiler-Menü nutzen), um nur diese Datei zu bauen.
- 5** Im **Compiler-Output**-Panel unten nachsehen – dort stehen die genauen Compiler-/Linker-Befehle und ob der Build erfolgreich war.
- 6** Optional auf **Start Emulator** klicken, um UAE zu starten und das frisch gebaute Programm gleich auszuprobieren.

Hinweis: Die Ad-hoc-Kompilierung baut und linkt immer nur die eine geöffnete Datei. Falls dein Programm mehr als eine Quelldatei braucht, wechsle stattdessen zur projektgesteuerten Kompilierung (siehe nächstes Kapitel).

Modus 2: Projektgesteuerte Kompilierung

Nutze diesen Modus für alles mit mehr als einer Quelldatei, für eine GUI (ReAction oder MUI), oder für Programme, die du über längere Zeit weiterbauen und pflegen willst.



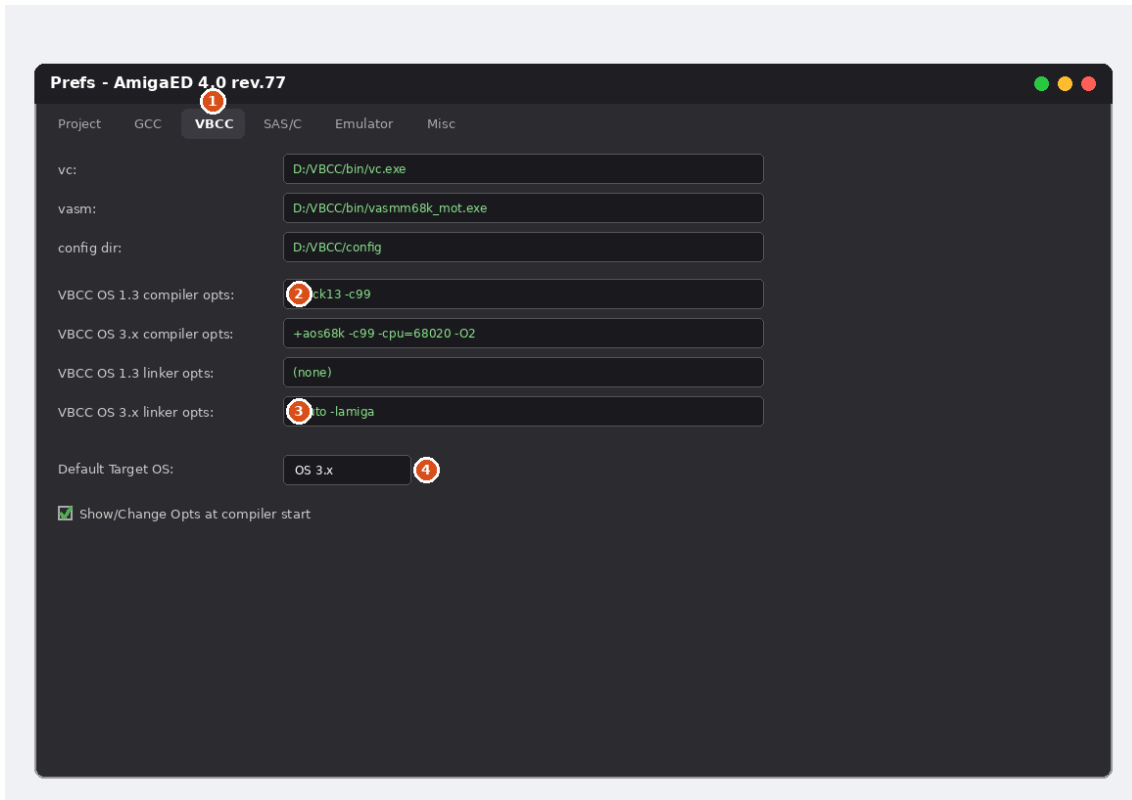
Das Projekt-Panel und die Toolbar: Die Nummern entsprechen den Schritten unten.

- 1 File** → **New Project...**, um mit einer Vorlage zu starten (Empty C, AmigaShell, AmigaOS 1.3, AmigaOS 3.x, ReAction oder MUI), oder **File** → **Import existing Project...**, um einen Ordner mit Quelldateien einzubinden, den AmigaED noch nicht kennt.
- Das **Projekt-Panel** links zeigt alle Dateien, automatisch in Kategorien sortiert – C/C++-Quellen, Header, Assembler-Quellen, AmigaGuide, Installer-Skripte, Executables, Sonstige Dateien, sowie die automatisch erzeugten Makefiles.
- Jederzeit weitere Dateien über **Add files to Project...** hinzufügen, oder sie einfach per Drag & Drop auf das Projekt-Panel ziehen.
- Auf **Build Project** klicken. AmigaED erzeugt die Makefiles für jede konfigurierte Toolchain (neu) und startet den Build.
- Das **Build-Output**-Panel beobachten. Bei Erfolg erscheint das gebaute Programm im Projekt-Panel in der Kategorie **Executable**, zusammen mit seiner Dateigröße.
- Auf **Start Emulator** klicken, um UAE zu starten und das Programm zu testen.

Hinweis: Die Makefiles eines Projekts werden automatisch erzeugt, sobald eine Datei zum Projekt hinzugefügt oder daraus entfernt wird – von Hand schreiben oder bearbeiten musst du sie nie. Es werden gleichzeitig sowohl vbcc- als auch gcc/g++-Makefiles erzeugt, dazu ein SAS/C-Makefile für den späteren Bau auf einem echten Amiga.

Der Einstellungs-Editor (Prefs)

File → Prefs... öffnet AmigaEDs Einstellungen, gegliedert in Reiter: **Project** (Autor/E-Mail/Website, Standard-Projektordner), **GCC**, **VBCC** und **SAS/C** (Pfade zu jeder Toolchain sowie deren Compiler-/Linker-Optionen – jeweils getrennte Felder für AmigaOS-1.3- und AmigaOS-3.x-Ziele), **Emulator** (Pfad zu deiner UAE-Programmdatei) und **Misc** (Anwendungsstil/Theme, Standard-Icon für neue Executables, sowie weiteres Editor-Verhalten).



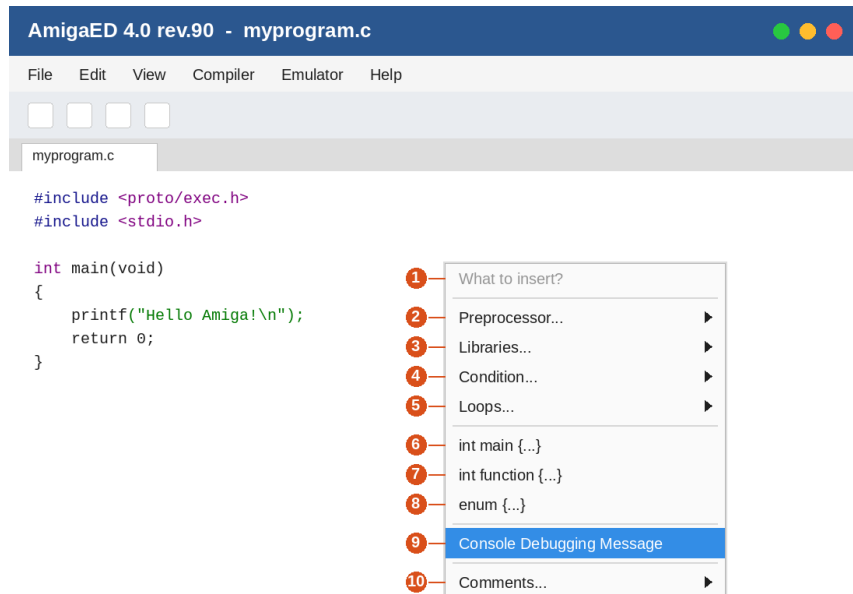
Der VBCC-Reiter als Beispiel – GCC und SAS/C folgen demselben Muster.

- 1 Die Reiter oben wechseln zwischen Project-, GCC-, VBCC-, SAS/C-, Emulator- und Misc-Einstellungen.
- 2 Compiler-Optionen werden **getrennt für AmigaOS 1.3 und AmigaOS 3.x** vorgehalten – die beiden Ziele brauchen üblicherweise unterschiedliche Schalter (siehe Tabellen unten).
- 3 Linker-Optionen sind ebenso pro Ziel-OS getrennt – hier wird automatisch eine Mathe-Bibliothek ergänzt, wenn dein Projekt Gleitkommazahlen verwendet (siehe Hinweis unten).
- 4 **Default Target OS** legt fest, welcher der beiden obigen Optionssätze verwendet wird, wenn du nicht ausdrücklich einen auswählst.

Automatische Gleitkommazahlen-Behandlung: Erkennt AmigaED irgendwo in den eigenen C/C++-Quellen deines Projekts `float` oder `double`, wird automatisch die passende Mathe-Bibliothek in den erzeugten Makefiles ergänzt – `-lm` für gcc/g++, `-lmieee` für vbcc, und `MATH=IEEE` für SAS/C (nur falls noch kein `MATH=-Modus` gesetzt ist). Das musst du nicht selbst ergänzen.

Das Kontextmenü des Editors

Ein Rechtsklick irgendwo im Code-Editor öffnet ein Kontextmenü mit schnellen Code-Einfüge-Hilfen. Es spiegelt exakt das Hauptmenü **Inserts** (Menüleiste) wider - alles, was hier aufgeführt ist, findest du dort unter demselben Namen wieder.



Das Kontextmenü des Editors, geöffnet über einer Quelldatei: Die Nummern entsprechen den unten erklärten Einträgen.

- 1 What to insert?** - eine deaktivierte Kopfzeile, kein anklickbarer Eintrag. Sie dient nur als Beschriftung des Menüs.
- 2 Preprocessor...** - ein Untermenü mit Präprozessor-Einfügungen: `#include`, `#define`, `#ifdef`, `#ifndef`, `#if defined(...)`, **Amiga #include files** (ein fertiger Block der gängigsten Amiga-NDK-Header), **Identify Amiga compiler** (eine Kette aus `#if defined(...)`-Prüfungen, die eine Konstante `compiler_string` mit dem Namen des jeweils verwendeten Compilers erzeugt), sowie **Amiga C version string** (ein `$VER:-`Versionsstring).
- 3 Libraries...** - `OpenLibrary()` und `CloseLibrary()` fügen je eine bearbeitbare `dummy.library`-Open/Close-Vorlage mit Fehlerbehandlung ein.
- 4 Condition...** - Vorlagen für `if(...)` {...} und `if(...)` {...} `else` {...}.
- 5 Loops...** - `while(...)` {...}, `for(...)` {...}, `do...{...}while(...)` sowie `switch(...)` (bereits mit einem `case dummy: break;`).
- 6 int main {...}** - fügt eine vollständige `main()`-Funktion ein; deaktiviert, sobald die Datei bereits eine enthält.
- 7 int function {...}** - fügt einen Funktionsprototyp plus passende Funktionsvorlage ein, bereit zum Ausschneiden und Einfügen an die richtige Stelle.
- 8 enum {...}** - fügt eine vollständige C-Enumeration ein (SomeEnum mit drei Beispielwerten).
- 9 Console Debugging Message** - fügt einen `if (myDebug) {...}`-Debug-Block ein; der Cursor landet direkt darin, bereit zum Tippen. Benötigt ein zuvor im Quelltext definiertes `#define myDebug TRUE` - das ist bereits am Anfang jeder "New Project"-Vorlage enthalten.
- 10 Comments...** - **Fileheader comment...**, C-style Einzel-/Mehrzeilkommentare, ein C++-Einzeilkommentar sowie ein C-artiger Trennlinien-Kommentar.

Hinweis: Die meisten Einträge fügen ihre Vorlage direkt an der Cursor-Position ein und positionieren den Cursor anschließend so, dass direkt weitergetippt werden kann - entweder in einer neuen Zeile nach dem eingefügten Block, oder (bei Console Debugging Message) direkt darin. Dieses Kontextmenü nutzt exakt dieselben Aktionen wie das Hauptmenü **Inserts** - beide sind daher immer synchron.

Empfohlene Compiler- & Linker-Schalter

Das sind AmigaEDs eigene, eingebaute Standardwerte – ein solider, gut erprobter Ausgangspunkt für jede Toolchain und jedes Ziel. Du kannst jederzeit eigene, projektspezifische Schalter in denselben Prefs-Feldern ergänzen.

m68k-amigaos-gcc (GCC/G++)

OS-1.3-Compiler	-Wall -O2 -mcrtnix13	Übliche Warnungen, Optimierung, Bindung gegen libnix, gebaut für Kickstart 1.3.
OS-3.x-Compiler	-Wall -O2 -noixemul	Übliche Warnungen, Optimierung, nutzt die AmigaOS-native C-Laufzeitumgebung statt ixemul.library.
OS-1.3-Linker	(keine)	Die OS-1.3-Vorlage ist reine Konsolenanwendung und braucht standardmäßig keine Zusatzbibliotheken.
OS-3.x-Linker	-lamiga	Für Workbench-fähige Programme (ReAction/MUI) nötig – wird von AmigaEDs OS-3.x-/ReAction-/MUI-Vorlagen automatisch ergänzt.

vbcc (vc)

OS-1.3-Compiler	+kick13 -c99	Benannte vbcc-Konfiguration für Kickstart 1.3, C99-Sprachmodus.
OS-3.x-Compiler	+aos68k -c99	Benannte vbcc-Konfiguration für AmigaOS 2.0+/3.x, C99-Sprachmodus.
OS-1.3-Linker	(keine)	Die reine Konsolen-OS-1.3-Vorlage braucht standardmäßig keine Zusatzbibliotheken.
OS-3.x-Linker	-lauto -lamiga	vbccs eigenes Äquivalent zu amiga.lib, nötig für Workbench-fähige (ReAction/MUI) Programme.

Optionale Ergänzungen, die viele Nutzer für einen Release-Build hinzufügen, sobald der Code sauber compiliert: -cpu=68020 (oder höher, falls du bewusst einen beschleunigten Amiga als Ziel hast – weglassen für maximale Kompatibilität mit einfachen 68000ern), -O2/-O3 (Optimierung), -size (auf Codegröße optimieren), -final (zur Fehlersuche gedachte Laufzeitprüfungen abschalten). Diese sind bewusst nicht Teil von AmigaEDs eigenen Standardwerten, da sie Kompatibilität bzw. Debugbarkeit gegen Geschwindigkeit/Größe eintauschen – erst gezielt ergänzen, wenn ein funktionierender Build bereits steht.

SAS/C

SAS/C läuft nur auf einem echten Amiga oder Emulator, AmigaED ruft es daher selbst nie auf – es erzeugt lediglich ein Makefile.sc zusätzlich zu den anderen, für die spätere, manuelle Nutzung dort. Das SCOPTS-Feld in den Prefs ist zunächst leer; AmigaED ergänzt trotzdem automatisch MATH=IEEE, falls dein Projekt das braucht (siehe oben).

Empfehlungen für Amiga-C-Programmierer

Eine kurze, handverlesene Liste an Literatur und Werkzeugen, die in der Amiga-C-/ReAction-/MUI-Entwicklergemeinde besonders empfohlen werden – nichts davon liegt AmigaED bei, aber alles passt gut dazu.

Literatur

Amiga ROM Kernel Reference Manual: Exec / Libraries and Devices / Includes and Autodocs

Die ursprüngliche Commodore-Amiga-Referenzreihe – nach wie vor die maßgebliche Quelle für Exec, Intuition und den Rest der klassischen System-Bibliotheken. Vergriffen; freie Scans sind auf archive.org archiviert.

Amiga ROM Kernel Reference Manual: AmigaDOS – Thomas Richter

Eine moderne, im RKRM-Stil gehaltene Referenz speziell für AmigaDOS/dos.library, die eine Lücke schließt, die die ursprüngliche Reihe nie vollständig abgedeckt hat. Kostenloser Download bei Aminet; eine gedruckte Ausgabe war zeitweise auch über lookbehindyou.de erhältlich.

ROM Kernel Reference Manual: Changes & Additions – Camilla Boemann & Jason Stead

Ein fortlaufendes (WIP-)Werk, das alles abdeckt, was dem AmigaOS von Release 2 bis 3.2 hinzugefügt wurde und was die ursprüngliche RKRM-Reihe zeitlich nicht mehr erfasst – gewidmet dem ursprünglichen RKRM-Autor Robert A. Peck. Kostenloses PDF von developer.amigaos3.net.

Erik Bartmanns Amiga Programmierbuch

C-Programmierung, Crosscompiling mit vbcc und ReAction, auf einem Amiga 4000 (bzw. Amiga Forever). Buch: 32€, E-Book: 19,99€, über bombini-verlag.de.

Erik Bartmanns Amiga 1200 Buch

Maschinensprache, C-Programmierung und Shell – 717 Seiten in 28 Kapiteln, mit Fokus auf den Amiga 1200. Buch: 64€, E-Book: 29,99€, über bombini-verlag.de.

Werkzeuge

Rebuild – Darren Coles

Ein ReAction-GUI-Builder (eine komplette Neuentwicklung des klassischen ClassMate), der C- oder Amiga-E-Code für Fenster, Menüs und Gadgets erzeugt. Public Domain. Quelle: github.com/dmcoles/ReBuild

Codecraft – Camilla Boemann

Codecraft ist eine leistungsstarke IDE für die native Softwareentwicklung auf dem Amiga. Codecraft macht es für dich als Entwickler einfach, Code zu schreiben und das daraus entstehende Programm zu bauen, auszuführen und zu debuggen. Und das alles ist in einer einzigen, einheitlichen Benutzeroberfläche griffbereit. Eine native IDE für AmigaOS 3.2.3 und neuer. Quelle: gitlab.com/boemann/codecraft, <http://boemann.dk/codecraft/>

MUIBuilder

Ein Oberflächen-Designer für die MUI- und Zune-Toolkits, mit dem Ziel, auf allen AmigaOS-artigen Systemen portabel zu sein. GPLv3/LGPLv3. Quelle: sourceforge.net/projects/muibuilder

Hinweis: Preise, Links und Verfügbarkeit (insbesondere gedruckter Auflagen) ändern sich mit der Zeit – sollte oben etwas nicht mehr erreichbar sein, findet eine Websuche nach Titel und Autor meist am schnellsten die aktuelle Quelle.